PROJECT DEEP DIVE / OCTOBER 2026

Agentic SRE

Interview report: architecture, agent workflow, security, testing, honest limitations and ready-to-use answers

The 60-second answer

I built a controlled SRE incident-response simulation using FastAPI, React and PostgreSQL. It generates service telemetry, detects incidents, runs four bounded investigation stages, proposes a simulated recovery action, pauses for an authenticated human approval, verifies recovery from fresh samples, and produces an audit trail and postmortem. A typed tool gateway and deterministic policy keep model narration away from execution decisions. The system is tested end to end, but it does not yet connect to real Kubernetes, Prometheus or a vector knowledge base.

What problem does it solve?

During an outage, an SRE must separate symptoms from likely causes, gather relevant evidence, choose a safe action and prove that service health returned. This project demonstrates that sequence in one inspectable workflow, with a human decision before any simulated state change.

The strongest interview story

The interesting engineering choice is the safety boundary: an LLM can explain bounded evidence, but it cannot choose the trusted target, authorize a write or declare recovery. Fresh telemetry, typed operations, role checks, persisted approval and verification decide those transitions.

Truth in one line

Demo-ready simulation; not a production Kubernetes remediation control plane. The blueprint describes a larger controlled-cluster target. The report distinguishes delivered behavior from that target throughout.

Technology at a glance

Experience	API / workflow	State	Delivery
React + Vite, JavaScript	FastAPI, Pydantic, NetworkX, optional Ollama	PostgreSQL, SQLAlchemy, Prisma migrations	Docker Compose, Kubernetes manifest, GitHub Actions

Evidence baseline: repository commit 044b32a; verification record dated 14 October 2026. Prepared 18 October 2026. Current live-hosting status is unverified.

INTERVIEW BRIEFING / 02

Purpose and architecture

A dashboard for SRE operators, backed by a bounded simulation engine and persistent records.

Users and the operating loop

A Viewer observes, an Operator can investigate and propose, and an SRE Lead or Admin can approve. The operator sees incidents, topology, agent traces, remediation plans, audit history, runbooks and evaluation results. Sign Up creates a read-only Viewer account; higher privileges require administrator provisioning.

Logical flow

Entry	Analysis	Decision	Evidence of outcome
React dashboard / REST / authenticated WebSocket	FastAPI detection + sequential agent stages + typed simulated tools	Persisted plan + Lead/Admin approval + simulated action	Fresh service/caller samples + PostgreSQL trace/audit + postmortem

Components and boundaries

Component	What it owns	Important constraint
Frontend	Login, Sign Up, incident console, approval, trace and postmortem views.	Browser tokens are held in memory; client displays simulated status.
FastAPI	Auth, telemetry, incidents, agents, plans, runbooks, audit, health endpoints.	Single process runs the simulator; one worker and replica.
Simulation	Service faults, golden signals and anomaly detection over a registered topology.	Telemetry and chaos state are process-local, not real monitoring.
PostgreSQL	Users, incidents, plans, audit events, runbooks, benchmark results, agent runs.	Prisma owns schema/migrations; SQLAlchemy handles Python persistence.
Optional Ollama	Structured explanation of triage/RCA evidence.	Unavailable model falls back to deterministic narration; model cannot execute a write.

Why these choices?

FastAPI/Pydantic make request and tool contracts explicit; React supports a live incident console; PostgreSQL gives transactional state and audit persistence. NetworkX models service dependencies. Prisma supplies versioned database migrations and a generated TypeScript client, while Python uses SQLAlchemy/psycopg. This split is pragmatic but requires schema alignment checks.

> **System boundary to say out loud**
> A Docker VPS can host the web app and its PostgreSQL database. It does not supply the real Kubernetes workloads, Prometheus metrics, logs, traces or durable distributed workflow required for the blueprint's full target.

INTERVIEW BRIEFING / 03

An incident from signal to postmortem

Explain the sequence with a concrete payment-service example.

Stage	What actually happens
1. Inject	An authorized lead injects a latency, crash or saturation fault into a registered service. Faults expire automatically.
2. Detect	The simulation loop emits metrics even with no dashboard connection. The detector creates or updates a stable incident for the degraded service set.
3. Triage	The triage stage classifies severity and blast radius from incident context; optional model text is explanatory only.
4. Investigate	Typed tools read simulated health, synthetic diagnostic logs and registered topology for a bounded set of services.
5. RCA	A heuristic ranks services with fresh heartbeats and threshold-crossing anomalies. No current anomaly yields NEEDS_EVIDENCE, not a plan.
6. Propose	The remediation stage creates a persisted, simulation-labeled plan and pauses at a human gate.
7. Approve + execute	A Lead/Admin approves; the engine records the decision, clears the injected fault and rechecks state. It never mutates a Kubernetes workload.
8. Verify + report	Five fresh samples, recent heartbeats, healthy service metrics and affected callers are required before success. A resolved incident exposes a factual postmortem.

Agent roles and state

The four specialized stages are **Triage, Investigation, Root Cause Analysis and Remediation**. They share a typed workflow state containing incident ID, evidence bundle, hypotheses, confidence, proposal, approval state and execution trace. Stage snapshots are persisted in **agent_runs**. The graph still runs sequentially; restart/resume and the approval/verification nodes are not a LangGraph workflow.

Most important failure path

If evidence is stale or absent, the workflow stops at **NEEDS_EVIDENCE**. If post-action health does not recover, the plan is **FAILED_VERIFICATION** and the incident remains open. The system does not invent a successful rollback.

> **Interview example**
> A payment-service latency injection raises p95 response time. The detector opens an incident; the agents inspect health, synthetic logs and topology; a lead approves the proposed simulated recovery; the engine clears the fault; five new cycles verify healthy metrics before the postmortem is available.

INTERVIEW BRIEFING / 04

Backend, database and API design

One PostgreSQL database

PostgreSQL is the only supported database. Prisma defines the versioned schema and runs **migrate deploy** during deployment. Python uses SQLAlchemy and psycopg for operational queries and transactions. The same database stores users, incidents, plans, audit history, runbooks, benchmark records and persisted agent runs.

Transactional safety

Incident/plan changes and their audit events commit in one transaction. A storage failure returns 503 rather than silently accepting a partial operation. The hash-linked audit ledger detects modification or a broken chain in stored records, although a database administrator could rewrite the entire chain or remove its tail without an external trusted checkpoint.

Representative APIs to know

Route / category	Interview purpose
/login, /signup; auth endpoints	Serve distinct entry screens; issue and validate authenticated sessions.
/api/v1/chaos/inject	Create a synthetic fault for a controlled demonstration.
/api/v1/telemetry/incident-candidate	Inspect the current detector candidate in the simulated environment.
/api/v1/ai/stream-investigation/{id}	Stream agent node starts/completions and the approval checkpoint.
/api/v1/remediation/plans/{id}/approve	Apply authenticated human approval, simulated execution and recovery verification.
/api/v1/incidents/{id}/postmortem	Retrieve the record-derived report after verified resolution.
/healthz, /readyz	Separate process liveness from database/simulator readiness.

The design tradeoff

A process-local simulator simplifies a reproducible demo and avoids writing to real infrastructure. The cost is limited scale and restart behavior: one worker/replica is required; transient simulation state is lost on restart even though incident and stage records persist. A real product would move alert deduplication, telemetry and workflow checkpoints to durable infrastructure.

> **Likely question: Why Prisma and SQLAlchemy together?**
> Prisma provides versioned migrations, schema validation and a TypeScript client; SQLAlchemy handles Python runtime persistence. I would keep them aligned with migration checks and avoid claiming one library is the runtime ORM for both languages.

INTERVIEW BRIEFING / 05

What is actually agentic here?

Bounded stages, typed evidence and a human decision - with precise claims about the model.

Decision authority

The orchestrator executes four named stages with explicit shared state and trace steps. The optional Ollama adapter can produce structured narration for triage and RCA. If it is offline or returns invalid JSON, deterministic text remains available. Trusted severity, target ranking, confidence and the remediation gate come from application logic and current evidence, not free-form model output.

Tool and prompt boundaries

The tool gateway exposes named, validated operations such as service health, synthetic logs and topology lookup. It does not expose arbitrary shell commands. Incident text and logs are serialized as untrusted evidence and common token/password patterns are redacted before model prompts. The app validates model output and rejects a root-cause name outside the known blast radius.

RAG: answer honestly

Operators can save runbooks in PostgreSQL and search them by lexical term overlap; results have document IDs and excerpts. The current investigation graph does not retrieve those chunks. There is no embedding model, pgvector index, semantic retrieval or grounding evaluation. Call this **runbook search**, not vector RAG.

Measurement: separate proof from proxy

Evidence available	What it supports	What it does not support
Three full simulated E2E tests	Crash, latency and resource saturation traverse injection, agents, approval, verification and postmortem.	No real Kubernetes or observability adapter exercised.
Five isolated benchmark scenarios	Cause/action match and harness wall-clock durations can be measured.	Durations are not production MTTD/MTTR; approval and unsafe-action rate are unmeasured.
Agent trace and audit	Stage status, tool summaries and decisions are inspectable.	No durable graph resume, full tool-result references or prompt/model version provenance.

If asked how to improve it

First add read-only adapters to a controlled demo cluster with source IDs, timestamps and quotas. Then introduce durable graph/checkpoint execution and evidence-linked hypotheses. Next ingest versioned runbooks into pgvector and evaluate retrieval relevance, action validity, approval safety and verified recovery with a labeled scenario set.

> **Defensible statement**
> I designed a safe, inspectable multi-stage agent workflow in a simulator. I would not present it as an autonomous LLM-operated production SRE system.

INTERVIEW BRIEFING / 06

Security and DevOps

Controls that exist today, plus operational limits you should volunteer.

Identity and approval

Passwords are hashed; authenticated JWT requests are checked against current database permissions. Self-registration only grants Viewer. A proposal may come from an Operator, while approval requires Lead/Admin. A write is tied to a persisted plan and a concrete target; the present write only clears a simulation fault.

Defense-in-depth and limitations

Current control	Residual gap before real-cluster use
Explicit browser origins, security headers, non-root app image, private PostgreSQL port.	TLS reverse proxy, host hardening and external identity/MFA on an actual deployment.
Typed tools, bounded inputs, model-output validation, prompt redaction and human approval.	Per-environment allowlists, separate read/write identities, quotas, broad evidence redaction.
Persisted hash-chain audit and database backup script.	External signed checkpoints and routine encrypted off-host restore drills.
CI dependency audits, Bandit, frontend checks and Trivy image scans.	SBOM/provenance, action digest pinning, release policy and continuous patch handling.

Deployment path

A version tag runs CI before publishing application and Prisma-migration images to GHCR. The Docker-host bundle pulls those images, validates Compose, backs up an existing database, applies migrations, starts one API replica and checks readiness plus login/signup routes. The API listens on host loopback; an HTTPS reverse proxy and real domain are needed for public access. There is no validated live VPS deployment yet.

Failure and recovery behavior

PostgreSQL readiness blocks the API, Prisma migrations run before startup, and **/healthz** and **/readyz** serve separate checks. Migrations are forward-only; take a verified backup before schema upgrades. The documented backup script validates archive readability, but a separate-database restore rehearsal is still needed. Recreate rollout and process-local state mean a brief outage on upgrade.

> **Security answer to memorize**
> The strongest boundary is that no model text can directly execute a write. Today this is enforced for simulated actions. Real Kubernetes execution would require environment-scoped credentials, allowlists, stronger approval binding and independent verification before enabling it.

INTERVIEW BRIEFING / 07

Verification and interview demo

Use measurable evidence, and label the date and environment of every result.

Verified repository baseline

Check	Recorded result on 14 October 2026
PostgreSQL-backed backend suite	106 tests passed, including three full simulated incident paths and stale-evidence gating.
Postman/Newman-style API run	34 requests, 60 assertions, zero failures against a disposable PostgreSQL database.
Frontend and schema	Lint/build passed; Prisma validation, migration diff and client connection passed.
Security and delivery	Bandit/flake8/npm checks passed. Corrected GitHub CI passed Python audit, both Trivy image scans and Docker Compose smoke.

Those are dated verification results, not a claim that a public deployment or live cluster works today. The source record is **docs/verification-2026-10-14.md**.

A five-minute live demo script

Minute	Show	Say
0-1	Login as a Lead; dashboard/topology.	These are registered simulated services and role-controlled views.
1-2	Chaos: inject payment-service latency.	Fault injection is bounded and expires; it does not touch a real cluster.
2-3	Open incident; run investigation; inspect four stages.	Health, synthetic logs and topology feed the stages; no fresh anomaly means no proposal.
3-4	Inspect persisted plan and approve.	A human gate separates diagnosis from the simulated state change.
4-5	Show verification, postmortem and audit.	Five fresh samples determine success; record-derived report preserves uncertainty.

If the demo fails

Check **/readyz** first, then PostgreSQL, migration and API logs. Confirm a supported single worker/replica setup and fresh simulator heartbeat. If optional Ollama is unavailable, the deterministic fallback should still let the workflow proceed. Never substitute a prewritten success claim for an unsuccessful verification.

> **Good interviewer question to invite**
> What would you change before letting this product act on a real cluster? Answer with source attribution, durable state, least-privilege tools, approval binding, independent verification and staged rollout.

INTERVIEW BRIEFING / 08

Interview questions: architecture and AI

Concise answers you can adapt rather than recite word for word.

1. Why did you build this project?

Incident response combines noisy telemetry, causal reasoning, risky actions and post-action verification. I wanted to demonstrate the full decision path and make its safety gates inspectable.

2. Is this really a multi-agent system?

It has four specialized, stateful stages with a persisted trace and different responsibilities. They execute sequentially. I would call it a bounded multi-stage agent workflow, not a fully autonomous distributed agent system.

3. Does the LLM decide the root cause or action?

No. Ollama optionally narrates bounded triage/RCA evidence; the service ranking and action gate remain deterministic. Invalid or unavailable model output falls back safely.

4. How do you prevent prompt injection?

Incident/log strings are explicitly treated as untrusted data, common credentials are redacted before prompts, output is schema-validated, and a model-provided service must belong to the known blast radius. Crucially, model text cannot authorize a write.

5. What makes the tool gateway safe?

It exposes named typed operations and no arbitrary shell. The current tools read simulator state. Before a real adapter, I would add environment and namespace allowlists, quotas, identities and output provenance.

6. Is there RAG?

There is PostgreSQL-backed runbook storage and lexical citation search, but no vector RAG in the agent graph. pgvector and retrieval evaluation are planned work.

7. How do you handle missing or stale evidence?

RCA requires a recent heartbeat and an actual anomaly. If no current anomaly supports a proposal, the workflow records NEEDS_EVIDENCE and stops before remediation.

8. Can the workflow recover after restart?

Stage snapshots persist in agent_runs, but complete checkpoint/resume semantics are not implemented. The simulator state itself is process-local. Durable orchestration is a key production gate.

9. Why use a graph/topology?

Dependencies help show blast radius and distinguish a failing dependency from healthy callers. The current registered topology supports simulated reasoning; real traces and deployment history would improve causal confidence.

10. What metric proves success?

In the simulator, a plan succeeds only after five fresh healthy samples plus healthy callers. The benchmark also measures cause/action match. It does not establish production MTTR, grounding or unsafe-action rates.

INTERVIEW BRIEFING / 09

Interview questions: operations and next steps

11. Is it production ready?

It is ready for a controlled single-replica demonstration after a real host/TLS/backup setup. It is not ready to remediate a production Kubernetes cluster: live adapters, durable state, policy scope and staging evidence are still missing.

12. Why only one replica?

Simulation telemetry and faults live in process memory. Two replicas would disagree. I use one worker/replica and a Recreate rollout; scaling requires moving state and scheduling out of process.

13. How is approval enforced?

An authenticated Lead/Admin approves a persisted plan tied to a target; the engine records the decision before simulated execution. A real-cluster version needs stronger time-bound binding to the exact action and evidence version.

14. What happens when verification fails?

The plan records FAILED_VERIFICATION, the incident stays open and the system reports failure. It does not claim an automatic rollback that never happened.

15. What would you build next?

A dedicated demo Kubernetes environment, read-only Prometheus/log/trace/Kubernetes adapters with source IDs, recoverable graph state, then versioned pgvector runbooks and safety/evaluation gates. Real write credentials come last.

A strong STAR story

Situation: An incident demo could claim recovery from stale or missing data. **Task:** Make the investigation and approval path defensible. **Action:** Gate RCA on fresh anomalies, stop with NEEDS_EVIDENCE, persist stage traces, and require fresh service/caller samples after a human-approved simulated action. **Result:** Three full simulated E2E incident cases and stale-evidence regression tests pass in the dated verification record. **Honest limit:** These tests do not validate a real cluster.

Resume wording you can defend

Two concise bullets

Built a FastAPI/React/PostgreSQL SRE simulation with four bounded investigation stages, typed telemetry tools, role-based human approval, recovery verification and a hash-linked audit trail.

Automated PostgreSQL-backed tests, API assertions, frontend checks, dependency/image scans and Docker Compose smoke checks; documented remaining Kubernetes, RAG and durability gaps.

Source trail for interview follow-up

Blueprint: **Agentic_SRE_Multi_Agent_Project_Blueprint.pdf**, especially pp. 2-7, 10-11, 15-16. Repository: **README.md**; **docs/blueprint-gap-analysis.md**; **docs/security-threat-model.md**; **docs/operations.md**; **docs/verification-2026-10-14.md**; **src/services/multi_agent_orchestrator.py**; **src/services/remediation_engine.py**; **tests/test_end_to_end_incidents.py**; **.github/workflows/ci.yml**.

Report note: the blueprint is the target specification; statements about implementation come from the repository snapshot. Recheck tests and hosting before quoting results as current.